

Wazuh SOC Sigma-Rules | Detection Lab | SETUP

MINISOC LAB SETUP DOCUMENTATION

1. VIRTUAL MACHINE AND NETWORK CONFIGURATION

First, I configured the foundational infrastructure using VirtualBox. I provisioned two virtual machines: an Ubuntu server acting as the Wazuh Manager (SIEM) and a Windows 10 Pro machine acting as the victim endpoint.

To ensure the machines could communicate securely while still accessing the internet for updates, I configured dual network adapters for both VMs. Adapter 1 was set to a Host-only Adapter to create an isolated local network, and Adapter 2 was set to NAT for outbound internet access. The Ubuntu server was assigned the IP 192.168.56.105, and the Windows VM was assigned 192.168.56.104.

Network

Adapter 1: Intel PRO/1000 MT Desktop (Host-only Adapter, 'VirtualBox Host-Only Ethernet Adapter')
Adapter 2: Intel PRO/1000 MT Desktop (NAT)

1. WAZUH SERVER INSTALLATION (UBUNTU)

With the network established, I logged into the Ubuntu VM to install the central Wazuh components. I utilized the official Wazuh automated installation script. Using the curl command, I downloaded the wazuh-install.sh script and executed it with the -a flag to perform a complete, all-in-one installation of the manager, indexer, and dashboard.

```
jbuntu@ubuntu-VirtualBox:~$ curl -sO https://packages.wazuh.com/4.11/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
30/04/2026 07:14:16 INFO: Starting Wazuh installation assistant. Wazuh version: 4.11.2 (x86_64/AMD64)
30/04/2026 07:14:16 INFO: Verbose logging redirected to /var/log/wazuh-install.log
30/04/2026 07:14:34 ERROR: Wazuh manager already installed.
30/04/2026 07:14:34 ERROR: Wazuh indexer already installed.
30/04/2026 07:14:34 ERROR: Wazuh dashboard already installed.
30/04/2026 07:14:34 ERROR: Filebeat already installed.
30/04/2026 07:14:34 INFO: If you want to overwrite the current installation, run this script adding the option -o/--overwrite. This
will erase all the existing configuration and data.
jbuntu@ubuntu-VirtualBox:~$
```

Once the installation finished, I needed to confirm the platform was stable. I used systemctl status commands to verify that the wazuh-indexer, wazuh-manager,

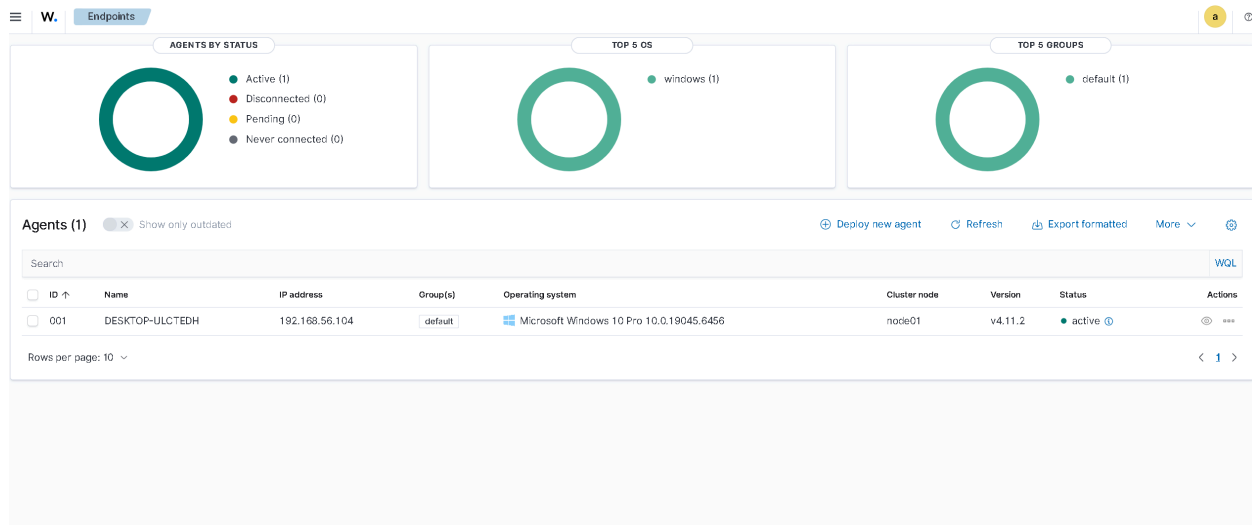
[illegible]

Next, I moved to the Windows 10 Pro VM to set up the telemetry collection. I opened PowerShell as an Administrator and used `Invoke-WebRequest` to pull the Wazuh agent installer directly from the Wazuh packages repository. I immediately executed the MSI installer, specifically pointing the `WAZUH_MANAGER` variable to the Ubuntu server's IP (192.168.56.105). After installation, I started the Wazuh service manually.

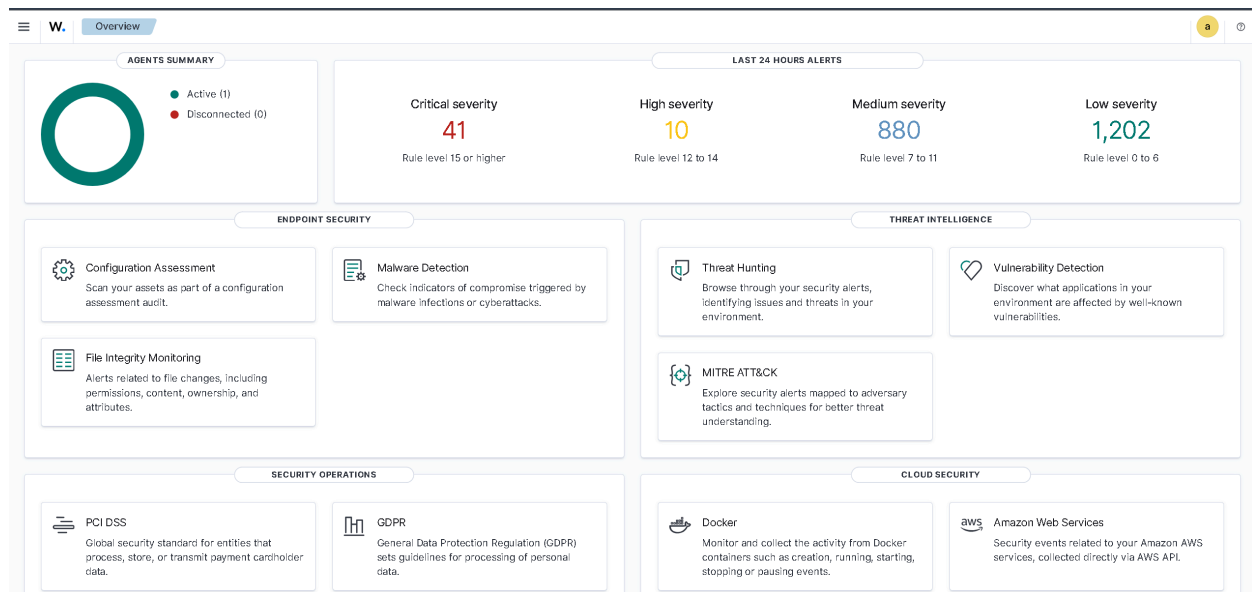
```
PS C:\Windows\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.11.2-1.msi -OutFile $env:tmp\wazuh-agent; m
Exec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.56.105' WAZUH_AGENT_NAME='Plo2w1st'
PS C:\Windows\system32> NET START WazuhSvc
The Wazuh service is starting.
The Wazuh service was started successfully.
PS C:\Windows\system32>
```

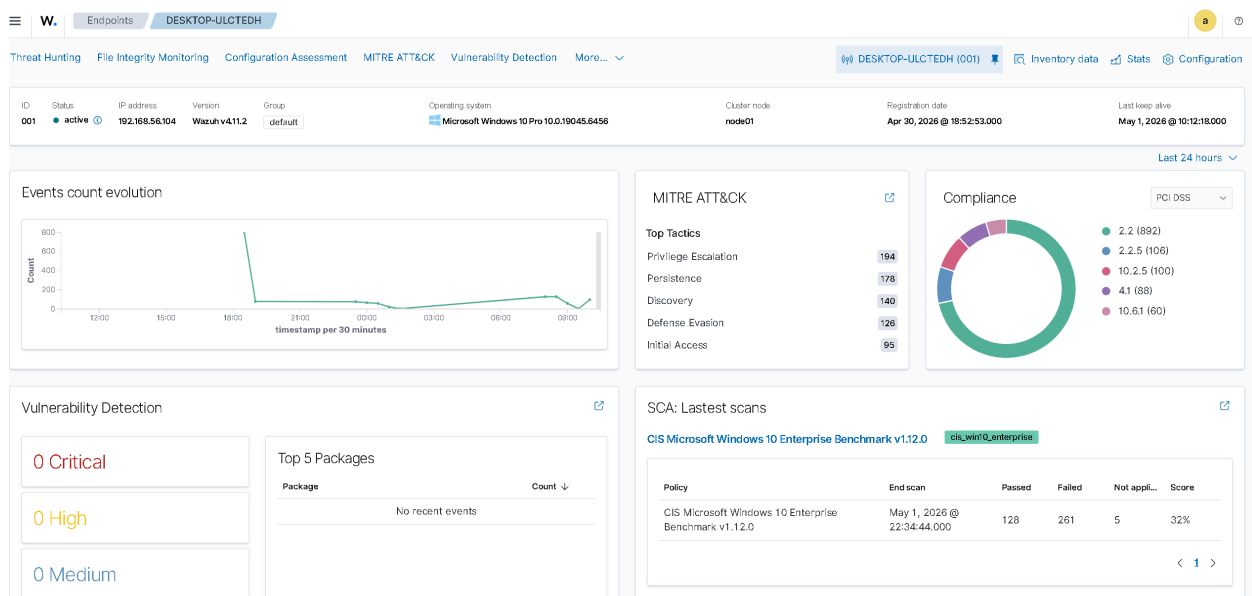
To prove the pipeline was working, I accessed the Wazuh web interface from my browser. I navigated to the Agents section to ensure the manager was receiving a heartbeat from the Windows VM. The dashboard successfully

listed the Windows 10 Pro agent as Active and connected via the 192.168.56.104 IP address.



I then navigated to the main overview and MITRE ATT&CK dashboards. This confirmed that the manager was not just connected, but actively parsing logs, triggering baseline rules, and mapping events to the MITRE framework.





1. REPOSITORY AND FOLDER ORGANIZATION

To finalize the project for deployment to GitHub, I structured the repository logically. I created a main directory containing three distinct folders to separate the lab configuration, the detection logic, and the execution proofs.

```

MiniSOC-Active-Defense
├── Setup # Detailed lab configuration and networking guides
├── sigma-rules # Raw, portable Sigma rules for SIEM deployment
└── Attack & Detection # Screenshots and logs proving the alerts fired
    ├── powershell nop # PowerShell obfuscation attack proofs
    ├── volume shadow copy # Ransomware precursor attack proofs
    ├── attack recon # Post-compromise discovery proofs
    ├── schedule tasks # Persistence mechanism proofs
    └── wevtutil # Defense evasion (log clearing) proofs
  
```

Inside the Attack & Detection folder, I created specific subdirectories dedicated to each mapped attack technique. This keeps the logs and screenshots organized for the PowerShell obfuscation, volume shadow copy deletion, system reconnaissance, scheduled tasks, and event log clearing demonstrations.

-ThankYou